

Deep Monkey Studio：面向工业数字孪生构建、运行与交付的一体化平台

张文鹏（Wenpeng Zhang）

独立研究者

15184552744@163.com

技术报告中文阅读版 • 2026 年 10 月 9 日

代码：<https://github.com/fatasia/DeepMonkey-Studio>

摘要

工业场景不仅需要显示几何，还需要连接设备状态、接受用户与智能体的操作，并在模型更新后继续运行。本文介绍 Deep Monkey Studio，一个以模块化图形引擎和可编程场景为核心的工业数字孪生平台。架构围绕三类接口组织：保留身份与保真范围的工程资产接口，连接编辑与领域动作的场景接口，以及描述资源、着色器和交付产物的执行接口。Deep Engine 在这一结构中提供 WebGPU 渲染、渲染图调度、预算约束下的流式资源驻留和浏览器/原生执行路径；平台在其上提供模型导入、工业协议、二维与三维编辑、脚本、插件、智能体和仿真。本文展开这些接口的状态归属与设计取舍，报告 86 项针对性测试以及已有引擎对照。在 1,000 对象场景中，Deep WebGPU 记录了 7 次绘制调用，但首帧延迟与动态帧时间高于部分对照。该系统展示了从异构输入到可编辑场景再到执行交付的组织方式，现有测量刻画其局部正确性与运行成本。

1 引言

工业模型、设备消息和交互场景具有不同的变化方式。工程文件按装配与设计版本变化，设备状态按时间变化，编辑结果则按用户意图变化。如果这些变化只在界面上汇合，重新导入、脚本操作或交付时仍可能丢失关联。平台设计的关键，是让这些对象在进入渲染之前就具有可供编辑和执行使用的身份与接口。

Deep Monkey Studio 采用可编程工业场景作为组织中心。场景连接工程资产、数据绑定和领域动作；引擎负责资源与图形执行；编辑器、脚本、插件和智能体通过各自接口操作场景；发布链将场景编译为运行时产物。这样的分工同时服务完整应用与引擎 SDK 使用，避免将平台能力绑定到单一编辑器或渲染后端。

本文的贡献是一个可查阅实现的系统设计：第一，以资产、场景和执行三类接口连接异构工业输入、创作与交付；第二，说明 Deep Engine 的渲染图、统一资源预算、候选资源发布和着色器包如何服务这一设计；第三，以功能覆盖、针对性软件行为与已有性能测量分析系统的实现范围和代价。电池算法在配套论文中讨论，本文将其视为领域服务的一个实例。

研究范围。 本文回答三个问题：平台怎样表示并连接工业场景的组成部分；引擎与创作工具之间怎样划分职责；已有实现与测量能支持哪些运行能力和性能判断。跨任务生产率与长期现场可靠性是后续评价对象。

2 相关工作

Unity 将游戏引擎视为可配置的智能体研究平台，讨论平台抽象、接口和研究能力 [8]。本文关注工业模型、设备连接与持续编辑的结合；平台级贡献由可复用的组织方式和运行能力体现，渲染算法只是其中一个层面。

Holodeck 通过资产与空间约束研究语言引导的具身环境生成 [1]。SceneCraft 将结构化场景生成与程序化工具结合 [2]；RoboGen 将生成的任务和环境连接到机器人技能学习 [3]。Code2Games 将场景和玩法表示与引擎反馈、行为反馈联系起来 [4]。这些工作为本文采用显式表示和可执行评价提供了参考。

已有数字孪生软件也不局限于可视化演示。Eclipse Ditto 提供设备状态中间件 [5]；iTwin.js 提供基础设施数字孪生开发库 [6]；OpenTwins 结合了三维可视化、物联网数据流和机器学习预测 [7]。本文从输入保真、可编程操作、状态归属和交付依赖比较架构角色；现有结果没有提供这些平台之间的统一任务排名。

3 平台架构

3.1 三类接口与状态归属

图 1 展示系统的主要路径。资产接口回答“输入是什么、来自哪里、保留了什么”；场景接口回答“对象如何组织、绑定和修改”；执行接口回答“运行时需要哪些资源与配置”。这三类接口是对现有实现的归纳，不要求子系统共享一个全局数据库或事务。

用 $A = (i, r, p, g, m)$ 表示概念上的资产记录，其中 i 是包身份， r 是修订， p 是支持配置， g 是运行时几何， m 是保留的层级、属性和来源信息。场景对象保存资产引用、变换、覆盖设置和绑定。重新导入产生新的资产修订，由场景适配器决定受影响的引用与可保留的覆盖。渲染资源身份和本体目标身份通过相应适配器使用，不能仅因标识符名称相同就认为它们具有相同生命周期。

场景写入、智能体检查点、物理会话与发布候选分别拥有状态。跨层操作传递目标身份、版本或内容哈希，接收方在自己的边界进行检查。该设计允许各层独立演进，也意味着局部回滚与回执不能自动升级为分布式恰好一次保证。

层次	职责与接口
输入	工程文件、设备协议、数据库查询和事件流；源身份与格式能力配置。
资产与数据构建	带版本的模型包、层级与属性、规范化数据行和事件、对象绑定。 二维与三维编辑器、仪表板、脚本、插件、场景事务，以及人与智能体使用的工具。
运行	WebGPU 渲染、本体动作、持久化智能体任务、无界面物理仿真及领域服务。
交付	编译后的场景产物、运行时包、浏览器与原生消费方、内容哈希及运行证据。

实现结合了 TypeScript/React 应用代码与 Rust/原生组件。现有 Three.js 查看器与 Deep WebGPU 后端并存。与引擎无关的身份标识和版本化资源包连接构建与执行；各适配器都有明确的支持范围。

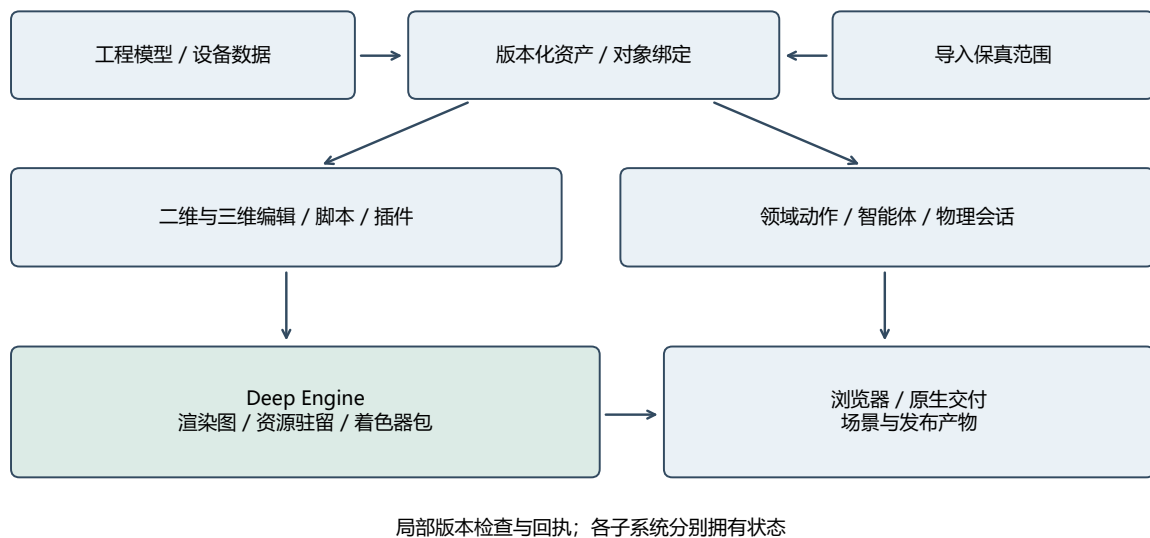


图 1: Deep Monkey 的输入、创作与执行路径。箭头表示接口连接；各子系统保留自己的状态归属。

3.2 Deep Engine: 渲染、资源与跨端执行

Deep Engine 提供资源包、实例化、渲染调度、着色器包和 WebGPU 执行。渲染实现包括 GGX 材质、基于图像的光照、方向光阴影、线性 HDR、色调映射、透明处理和后处理。glTF 路径处理受支持的纹理通道、节点动画、骨骼蒙皮和形态目标；遇到未支持的编码特性时返回结构化错误。

资源驻留管理对几何和纹理使用统一预算。可见块与预取块按需请求细节层级；候选资源先经过校验，再在帧边界发布。取消请求和被新请求替代的请求分别控制待执行上传与清理。着色器包记录入口点、绑定、顶点布局和渲染通道配置，并使用 TypeScript/Rust 共享表示。版本化二维显示列表将路径、文字、图片、裁剪与命中区域传递给原生图表渲染。

这些机制支撑本地交付与后端演进。整个产品的视觉等价性、设备丢失恢复，以及完整动画和物理能力覆盖，需要独立于隔离渲染基准开展验收。

调度与资源复用。 渲染图检查资源依赖并按拓扑次序执行，同时生成临时资源的生命周期和物理槽计划。跨资源复用要求格式、尺寸、采样数与使用方式相容，外部资源不进入临时槽。这使资源复用成为显式调度决策。大场景驻留规划保留可见对象可绘制的最粗细节层，再按预算追加几何细节和纹理 mip；被新请求替代的上传通过所有权和清理规则退出。

可编程材质与执行接口。 DeepSL surface 编译路径将 Standard/Unlit 材质属性转换为类型化中间表示与 WGSL。版本化着色器包描述入口、绑定、顶点流、附件及渲染状态，由浏览器与 Rust 执行器校验并创建管线。缓存区分可持久化内容与设备相关对象。场景状态与后端分离，宿主控制绘制节奏和后端切换，编辑器、独立 SDK 与原生播放路径由此使用相应接口。

3.3 完整创作工作流与扩展

平台将二维页面、三维场景、数据绑定、脚本和插件组织在同一项目中。模型层级和属性用于查找与选择对象，设备数据为绑定对象提供状态，仪表板呈现相同项目的数据，领域服务为动作和

分析提供计算。引擎承担视觉与运行时执行，编辑器提供交互入口；场景事务和发布产物连接编辑与部署。这样的分工让渲染引擎可以作为模块使用，同时保留完整的创作应用。

功能完整性应按任务路径呈现：接入工程模型和设备数据、建立对象绑定、编辑场景与页面、执行领域动作、保存并交付。模型格式和协议的广度决定输入覆盖；脚本、插件和 SDK 决定扩展方式；运行时包和本地部署决定交付方式。下文分别列出已查阅的输入路径、操作与测量范围。

3.4 模型导入与保真

导入过程保留源文件信息，并生成附带层级和属性的运行时几何。格式覆盖以版本、支持子集和保留信息描述。表 1 总结了已检查的实现路径与文档，并非完整的兼容性认证。

表 1: 工程模型路径与支持范围。

格式	已实现路径或已验证范围
glTF / GLB	运行时交换、优化与发布；扩展支持范围由具体加载器决定。
IFC	通过 fragments 导入 BIM，保留源表示与语义访问能力。
STEP / STP	通过开源曲面离散化生成运行时几何，并附带层级与属性文件。
FBX; DXF	分别通过对应适配器完成场景与动画加载、二维图纸导入。
OpenUSD	支持已检查样本中的 USDA/USDC/USDZ 加载；复杂组合仍需单独验证。
JT	内置路径覆盖已检查的 9.5/10.3 配置中的 LOD0 三角几何、装配实例及部分属性与材质；PMI 结构解析与语义标注解析分开评价。
X_T	对限定几何子集提供内置文本解析。修剪、曲面类型、命名、颜色和装配信息的损失取决于具体配置。

更广泛的格式目录还记录了候选格式。目录中出现某格式，不代表导入器已经可用。原生专有格式须有通过准入的内置或开源实现路径，才能列入实际运行覆盖矩阵。评价应分别检查几何、装配结构、属性、交互和交付。

3.5 工业协议与数据源

API 调用实际的连接器适配器，并将返回数据规范化后提供给消费方。持续订阅与限定范围的预览读取满足不同的运行需求。

接口类别	已检查的操作
MQTT; OPC UA	持续订阅与预览读取; 共享订阅生命周期管理; 处理 OPC UA 质量、时间戳和配置的安全选项。
Modbus TCP; BACnet; S7; EtherNet/IP	在限定范围内读取寄存器、线圈、属性、PLC 地址或标签。
HTTP; WebSocket; Kafka; AMQP	通过各自适配器发起请求或进行限定范围的消息采样。
SNMP; CoAP; TCP; UDP; 串口	针对不同数据源的采样适配器。
SQL 与分析存储	PostgreSQL、MySQL 兼容存储、SQL Server、Oracle、ClickHouse 和时序数据源等查询适配器。
文档与文件数据源	MongoDB、Elasticsearch、CSV 和 Excel 适配器。

持续运行的 MQTT 与 OPC UA 数据源向事件基础设施提供数据。仿真结果可通过 MQTT 和 SQL 路径导出。实际设备覆盖取决于传输配置、地址映射、数据类型与厂商行为; 现场评价应同时记录这些条件、重连结果和事件顺序。

3.6 资产修订与受控编辑

源哈希与资源包身份将修订关联到具体资产。重导入协调器在发布前准备更新, 支持回滚、代次检查和冲突处理。产品适配器更新场景快照, 同时保留身份关系及适用的覆盖设置。

场景事务携带基准版本和经过校验的命令。事务层拒绝基于过期版本的写入, 并在部分执行、错误或取消后调用回滚。编辑器与智能体消费方共用该路径; API 与浏览器之间的桥接层保留事务身份并缓存结果。这为并发操作提供了明确的准入条件。

3.7 本体、智能体与仿真

本体动作将规范目标绑定到已发布的定义、参数、作用范围和前置条件, 执行后生成回执。属性约束分为仅报告和严格执行两种行为, 因此每次实验都需要记录所选模式。

已接入产品的工业智能体运行时使用工具网关、持久化检查点、取消机制和基于版本的恢复。Web 工作区提供恢复与取消操作。另一个独立的任务图辅助模块支持次数受限的修正, 但本次核查尚未找到它的产品消费方。

World API 在基于 Rapier 的无界面运行时上提供重置、步进、观测、快照、恢复和关闭操作。已检查的配置以 60 Hz 运行, 最多包含 256 个对象, 每次步进最多执行 600 个时间步。观测包含位姿、接触和事件; 相机、深度与激光雷达采样不在此配置范围内。参与刚体仿真的可视模型需要声明受支持的碰撞体。恢复过程检查快照完整性, 仅在重建成功后替换原会话。

4 评价

4.1 针对性软件验证

2026 年 10 月 9 日的验证共通过 9 个文件中的 86 项测试: 40 项世界运行时测试; 37 项 API 测试, 覆盖本体动作、智能体停止与恢复、World API 和编辑器事务桥; 5 项场景事务测试; 4 项任务图测试。测试检查了部分准入、重放、所有权、回滚和恢复行为。源码核查还追踪了相应的产品消费方。

4.2 已有渲染对照

10 月 7 日的基准在 i9-12900HX 与 RTX 4060 Laptop GPU 上运行，每种配置使用三个冷启动上下文；分辨率为 1440×900，DPR 为 1，启用 PBR、ACES 和 1024 分辨率阴影。表 2 选取包含 1,000 个对象和 614,294 个三角面的测试场景。帧时间 P95 为各轮浏览器 RAF 测量 P95 的中位数；GPU P95 来自时间戳测量。在这个场景中，Deep WebGPU 报告的绘制调用较少，静态帧时

后端	负载	首帧 ms	帧时间 P95 ms	GPU P95 ms	绘制调用
Three WebGL	静态	72.7	7.1	4.2	1055
Three WebGPU	静态	283.2	7.1	1.6	1056
Babylon WebGPU	静态	876.0	15.0	4.5	2001
Deep WebGPU	静态	1623.3	7.1	3.1	7
Three WebGL	动态	70.4	7.1	4.1	1055
Deep WebGPU	动态	1733.7	14.0	3.1	7

表 2: 已归档的隔离渲染测量。各引擎的绘制调用统计口径并不完全一致。

间具有竞争力。但其首帧出现明显更晚，动态帧时间也比 Three WebGL 参考实现更差。这些结果指出了启动和更新成本的优化方向，尚不能据此完成全产品视觉一致性判断或默认后端切换。

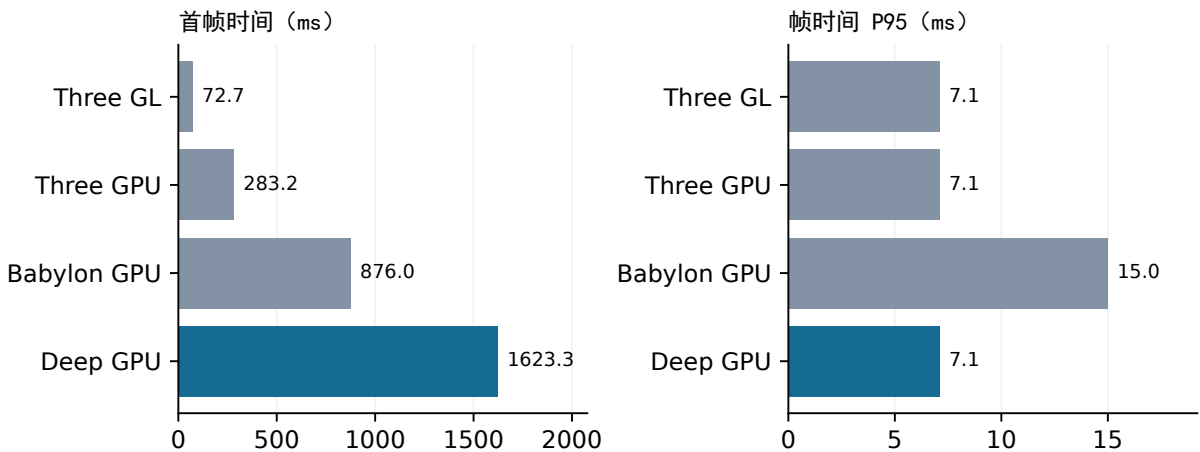


图 2: 已有对照中的 1,000 对象静态场景。启动与稳定运行的帧时间反映不同成本；数值越低越好。

4.3 结果解释

86 项测试支持所覆盖的输入检查、回滚、回放和恢复行为。渲染结果反映启动、稳定帧和动态负载的不同成本；较低的绘制调用数并未在该场景中带来全面的帧时间优势。因而本文将功能覆盖、正确性检查和性能分别报告。评估平台整体收益还需要固定资产和设备输入，完成模型更新、并发编辑、中断恢复与跨运行时交付的配对任务比较。

5 讨论与局限

平台的实用价值在于可编辑源码、本地运行路径、统一身份，以及跨工具检查状态变化的能力。这些特性允许在适用许可范围内定制和维护。商业成熟度还取决于持续现场负载、升级行为、操

作易用性和支持成本，本文尚未获得这些测量。

现有证据覆盖部分软件行为与隔离渲染。顺序回放回执不代表并发场景中动作恰好执行一次，事务原子性也依赖驱动实现。格式样本未覆盖所有版本和几何类型。完整生命周期对照是主要的后续实证工作。

6 可复现性与可用性

平台源码：<https://github.com/fatasia/DeepMonkey-Studio>。本轮实现查阅以提交 [8bbfd90535db](#) 为版本入口；历史渲染测量保留其原始版本。[Deep Engine 文档](#)说明引擎模块和运行边界，[部署指南](#)提供运行步骤。

配套证据记录列出了源码文件、已检查的产物、测试命令和后续实验。渲染记录使用其各自登记的版本与硬件，本文表格保留了它们与当前实现核查之间的区别。可复现发布应保留资产原始字节、能力配置声明、依赖与检查点版本、动作轨迹和预期结果。

Deep Monkey Studio 采用基于 MIT 修改的源码可用许可证，附加企业行为与人权条款，正式名称为 Deep Monkey Community Source License 1.0 (DMCSL-1.0)。具体条款以仓库许可证为准。论文与第三方素材分别适用各自的许可。

生成式 AI 辅助了源码检查、内容组织和论文起草。

7 结论

Deep Monkey Studio 以资产、场景与执行接口组织工业输入、创作和运行。Deep Engine 的图形、资源与着色器机制支撑模块化执行；模型接入、协议、编辑器、脚本、插件和领域服务提供完整应用路径。已有测试和测量为部分行为与运行成本提供了证据。系统设计的可复用价值在于把可编程工业场景落实为具体接口、状态归属和运行时产物；跨任务收益需要进一步配对评价。

参考文献

- [1] Y. Yang et al. Holodeck: Language Guided Generation of 3D Embodied AI Environments. CVPR, 2024. <https://arxiv.org/abs/2312.09067>.
- [2] Z. Hu et al. SceneCraft: An LLM Agent for Synthesizing 3D Scenes as Blender Code. ICML, 2024. <https://proceedings.mlr.press/v235/hu24g.html>.
- [3] Y. Wang et al. RoboGen: Towards Unleashing Infinite Data for Automated Robot Learning via Generative Simulation. ICML, 2024. <https://proceedings.mlr.press/v235/wang24cc.html>.
- [4] W. Wu et al. Code2Games: Enabling Coding Agents for Gaming World Generation. arXiv:2610.05033, 2026. <https://arxiv.org/abs/2610.05033>.
- [5] Eclipse Foundation. Eclipse Ditto: Overview. Accessed 9 October 2026. <https://eclipse.dev/ditto/intro-overview.html>.
- [6] iTwin.js. Getting Started. Accessed 9 October 2026. <https://www.itwinjs.org/learning/>.

- [7] J. Robles, C. Martín, and M. Díaz. OpenTwins: An open-source framework for the design, development and integration of effective 3D-IoT-AI-powered digital twins. arXiv:2301.05560, 2023. <https://arxiv.org/abs/2301.05560>.
- [8] A. Juliani et al. Unity: A General Platform for Intelligent Agents. arXiv:1809.02627. <https://arxiv.org/abs/1809.02627>.